On-Device AI 실습
Pruning for CNN

Dongkun Shin
Intelligent Embedded Systems Lab.
Sungkyunkwan University

Practice Setup

- **Download practice materials**
  - https://drive.google.com/drive/folders/1GZRDpshMbKn6f46Wdoj204tXMi4dOrcQ?usp=drive_link
  - Short Link: https://bit.ly/4fQXk3f

- **Open a command prompt or PowerShell in the directory containing the materials, set up the conda environment, and run Jupyter Lab**

```
(base) PS D:\실습> conda activate env_aias_test
(env_aias_test) PS D:\실습> jupyter lab
```

- **If you encounter environment issues, download the provided .bat file to perform a rollback**
  - ₩₩swschoolavdazfiles002.file.core.windows.net₩aias-language₩Env₩Rollback_env_aias_test.bat

Overview

1. **Pruning Granularity**
   - Learn and implement the differences between Fine-grained, Vector-level, Kernel-level, and Channel-level pruning

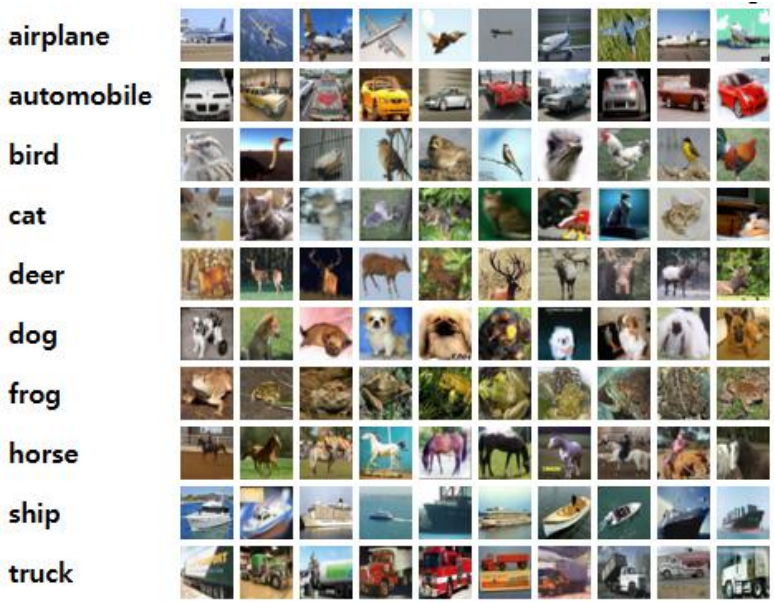
2. **Pruning Ratio**
   - Learn how to determine the sparsity (ratio of weights to remove) and check the difference between Layer-wise and Global methods through practice

3. **Pruning Schedule**
   - Understand One-shot and Iterative Pruning, and apply Linear and Cubic Scheduling to analyze performance changes

PyTorch Basic Tutorial: Dataset

- **A dataset is required to train a model; this practice uses the CIFAR-10 dataset**
- **In PyTorch, datasets are managed using a Python class called DataLoader**
  - It configures pre-processing (transforms), batch size, shuffling, and the number of workers
- **CIFAR-10 consists of 32x32 color images across 10 classes**
  - Train: 50,000
  - Test: 10,000

CIFAR-10 Dataset



https://www.cs.toronto.edu/~kriz/cifar.html

PyTorch Basic Tutorial: Dataset (Code snippet)

Data augmentation

```python
# 학습 및 테스트 데이터에 적용할 변환 함수
transforms_train = Compose([
    RandomCrop(image_size, padding=4),
    RandomHorizontalFlip(),
    ToTensor(),
])

transforms_test = Compose([
    ToTensor(),
])
```

```python
# CIFAR-10 데이터셋 로드
dataset_train = CIFAR10(
    root="data/cifar10",
    train=True,
    download=True,
    transform=transforms_train
)

dataset_test = CIFAR10(
    root="data/cifar10",
    train=False,
    download=True,
    transform=transforms_test
)
```

```python
# Dataloader 생성
dataloader_train = DataLoader(dataset_train, batch_size=512, shuffle=True, num_workers=0, pin_memory=True)
dataloader_test = DataLoader(dataset_test, batch_size=512, shuffle=False, num_workers=0, pin_memory=True)
dataloader = {"train": dataloader_train, "test": dataloader_test}
```

PyTorch Basic Tutorial: Model

In this practice, a pre-trained model is loaded using torch.hub

```
TORCH_HUB_REPO = "SKKU-ESLAB/pytorch-models"
MODEL_NAME = "cifar10_vgg9_bn"

model = torch.hub.load(TORCH_HUB_REPO, MODEL_NAME, pretrained=True)
if torch.cuda.is_available():
    model = model.cuda()

print(model)
```

PyTorch Basic Tutorial: Model Structure

- **In PyTorch, all models and layers are defined via the nn.Module class and organized hierarchically**

- **The practice uses a VGG9 model pre-trained on CIFAR-10**
  - consisting of 8 convolution layers and 1 linear layer

VGG9 Model Structure

```
VGG(
  (backbone): Sequential(
    (conv0): Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (bn0): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (relu0): ReLU(inplace=True)
    (conv1): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (relu1): ReLU(inplace=True)
    (pool0): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (conv2): Conv2d(128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (relu2): ReLU(inplace=True)
    (conv3): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (relu3): ReLU(inplace=True)
    (pool1): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (conv4): Conv2d(256, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (bn4): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (relu4): ReLU(inplace=True)
    (conv5): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (bn5): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (relu5): ReLU(inplace=True)
    (pool2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (conv6): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (bn6): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (relu6): ReLU(inplace=True)
    (conv7): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (bn7): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (relu7): ReLU(inplace=True)
    (pool3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (classifier): Linear(in_features=512, out_features=10, bias=True)
)
```

PyTorch Basic Tutorial: Evaluating Model Size

iterating through all parameters in the model

```python
def get_num_parameters(model: torch.nn.Module, count_nonzero_only=False) -> int:
    """
    모델의 총 파라미터 수 계산
    """
    num_counted_elements = 0
    for param in model.parameters():
        num_counted_elements += param.count_nonzero() if count_nonzero_only else param.numel()
    return num_counted_elements

def get_model_size(model: torch.nn.Module, data_width=32, count_nonzero_only=False) -> int:
    """
    모델의 크기를 비트 단위로 계산
    """
    return get_num_parameters(model, count_nonzero_only) * data_width

dense_model_size = get_model_size(model)
print(f"dense model has size={dense_model_size/ (1024 ** 2) / 8:.2f} MiB")
```

count the total number of elements that is not 0

count the total number of elements

PyTorch Basic Tutorial: Evaluating Model Accuracy

change the model to evaluation mode

```python
@torch.no_grad()
def evaluate(
    model: torch.nn.Module,
    dataloader: DataLoader,
    verbose=True,
) -> float:
    model.eval()
    num_samples, num_correct = 0, 0
    for inputs, targets in tqdm(dataloader, desc="Eval", leave=False, disable=not verbose):
        if torch.cuda.is_available():
            inputs, targets = inputs.cuda(), targets.cuda()
        outputs = model(inputs)
        preds = outputs.argmax(dim=1)
        num_samples += targets.size(0)
        num_correct += (preds == targets).sum()
    return (num_correct / num_samples * 100).item()
```

data is obtained via the DataLoader

output represents confidence values for the 10 classes, and the index of the largest value becomes the predicted class

output shape is (batch_size, num_classes)

PyTorch Basic Tutorial: Parameters

Layer	Parameters	Shape
torch.nn.Conv2d	weight	$\left(out_channels, \frac{in_channels}{groups}, kernel_height, kernel_width\right)$
	bias	$(out_channels)$
torch.nn.Linear	weight	$(out_features, in_features)$
	bias	$(out_features)$
torch.nn.BatchNorm2d	weight	$(num_features)$
	bias	$(num_features)$

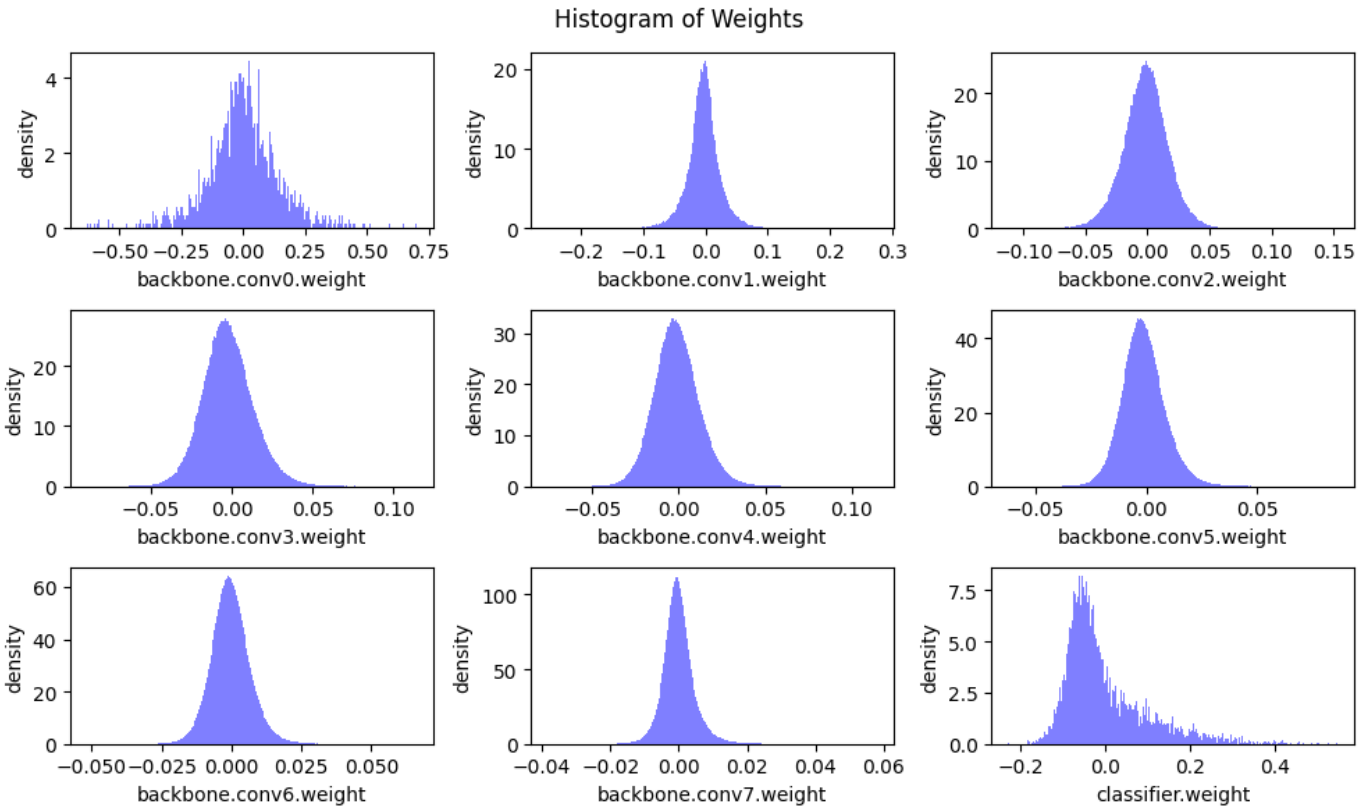
E.g., Conv2d와 Linear의 weight를 순회:

```
for param in model.parameters():
  if param.dim() > 1
    ...
```

or

```
for m in model.modules():
  if isinstance(m, torch.nn.Conv2d) or isinstance(m, torch.nn.Linear):
    param = m.weight
    ...
```

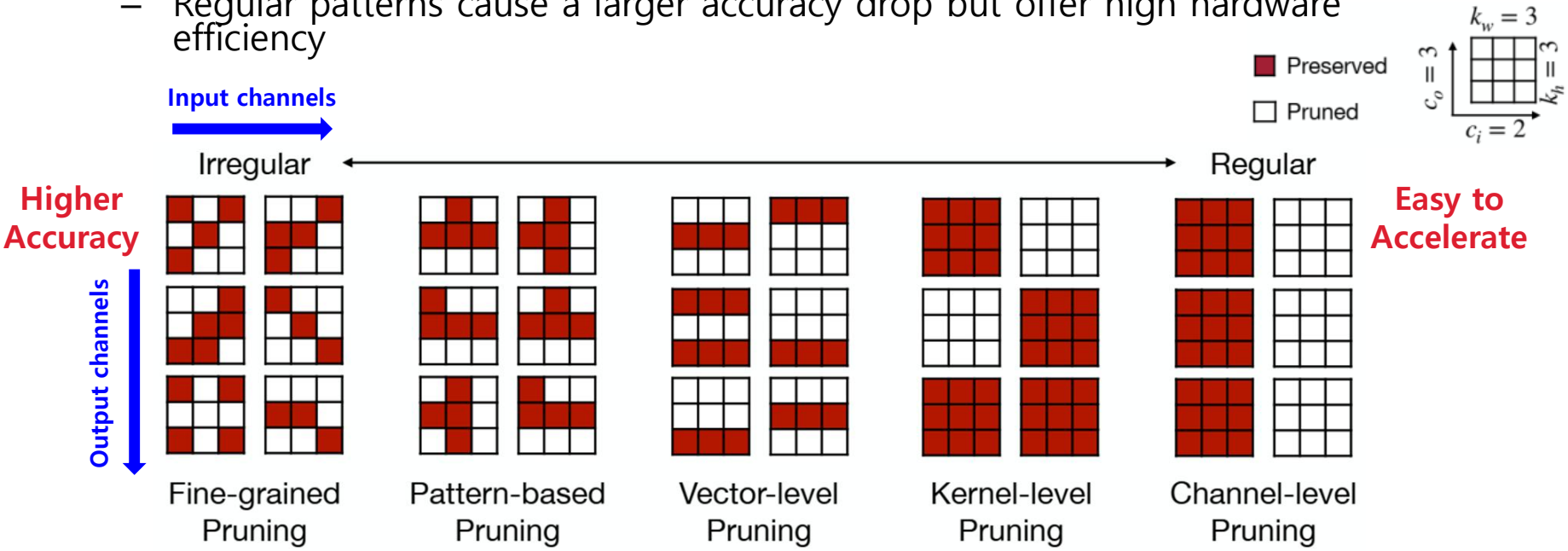
Check the weight distribution per layer of the pre-trained model



Many weights are distributed close to 0
➔ weights can be compressed by pruning

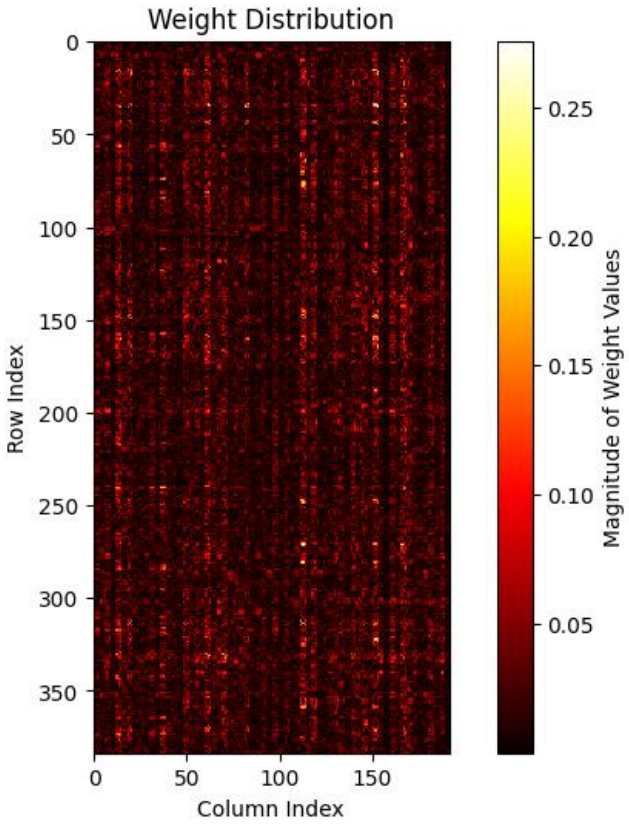
1.1. Pruning Granularity/Pattern

- **Write pruning code considering various pruning granularities**
  - Irregular patterns have a smaller drop in accuracy but lower hardware efficiency
  - Regular patterns cause a larger accuracy drop but offer high hardware efficiency

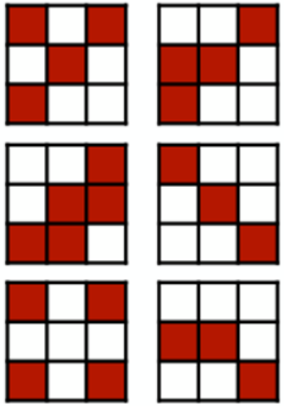


Print Weight Distribution

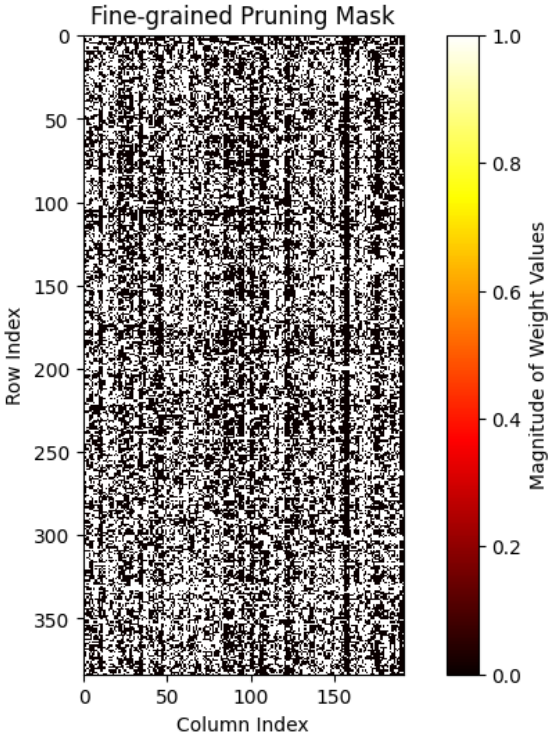
- **Right figure shows the magnitude of a (128, 64, 3, 3) weight**
  - The layout of convolution weights in PyTorch is (CO, CI, KH, KW)
    - CO: output channels
    - CI: input channels
    - KH: kernel height
    - KW: kernel width
  - For visualization, the layout was reshaped to (CO * KH, CI * KW) to arrange the data as shown in the previous slide



[실습 1] Fine-grained Pruning 구현



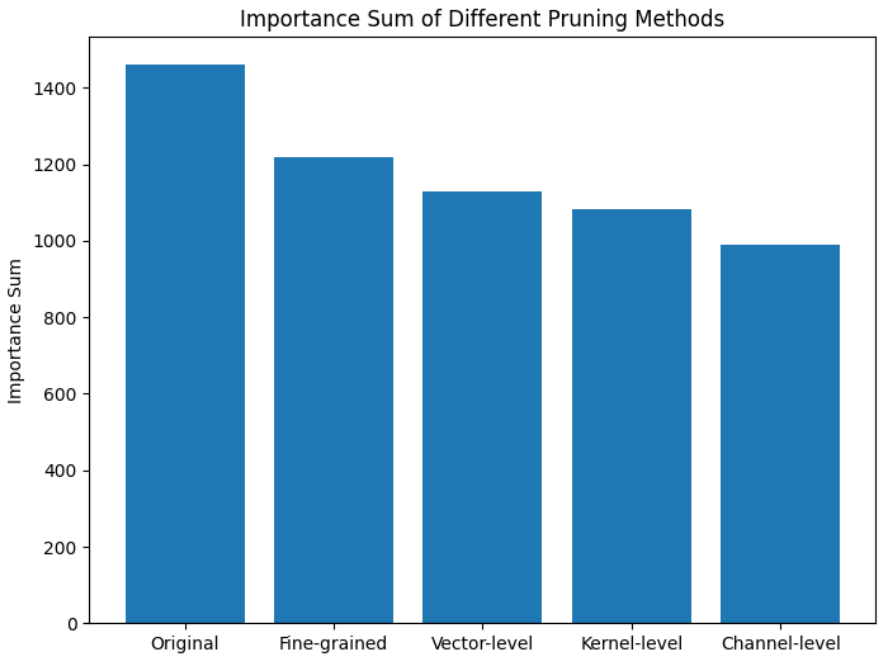
Fine-grained Pruning



[실습 1] Answer


```python
###################### YOUR CODE STARTS HERE ######################
# num_pruned_elements를 계산하세요.
# hint: round() 함수를 사용하세요.
num_pruned_elements = round(weight.numel() * sparsity)

# 절댓값을 사용하여 중요도 계산
# hint: torch.abs() 함수를 사용하세요.
importance = torch.abs(weight)

# pruning trheshold를 계산하세요.
# hint: torch.kthvalue() 함수를 사용하세요.
threshold = torch.kthvalue(importance.flatten(), num_pruned_elements)[0]

# threshold보다 큰 값들을 1로 설정
# hint: 부등호를 사용하세요.
mask = importance > threshold
###################### YOUR CODE ENDS HERE ########################
```

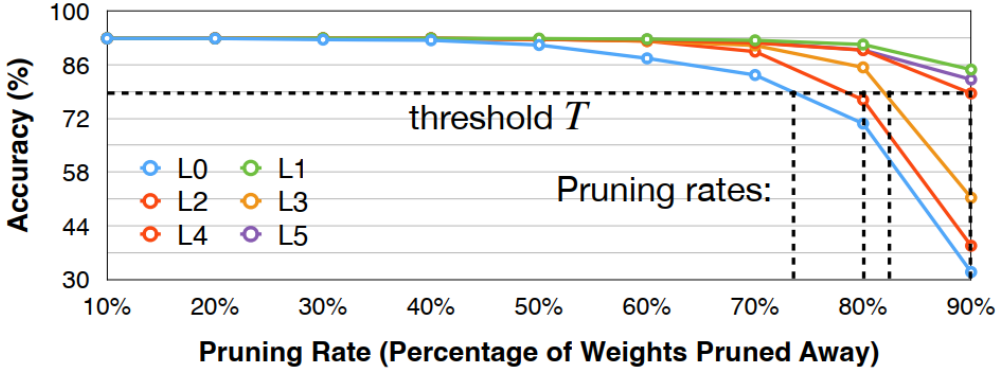
Compares the Importance Sum and Reconstruction Error across different pruning methods

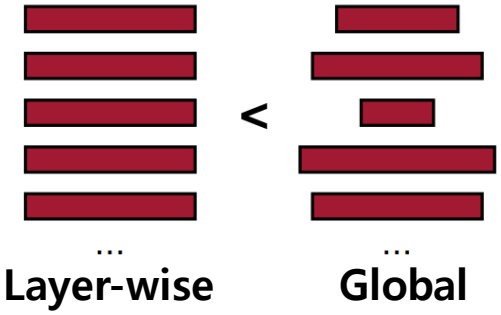


1.2. Pruning Ratio

- **Sensitivity Analysis**

Because each layer has a different sensitivity, different pruning ratios are required for each layer



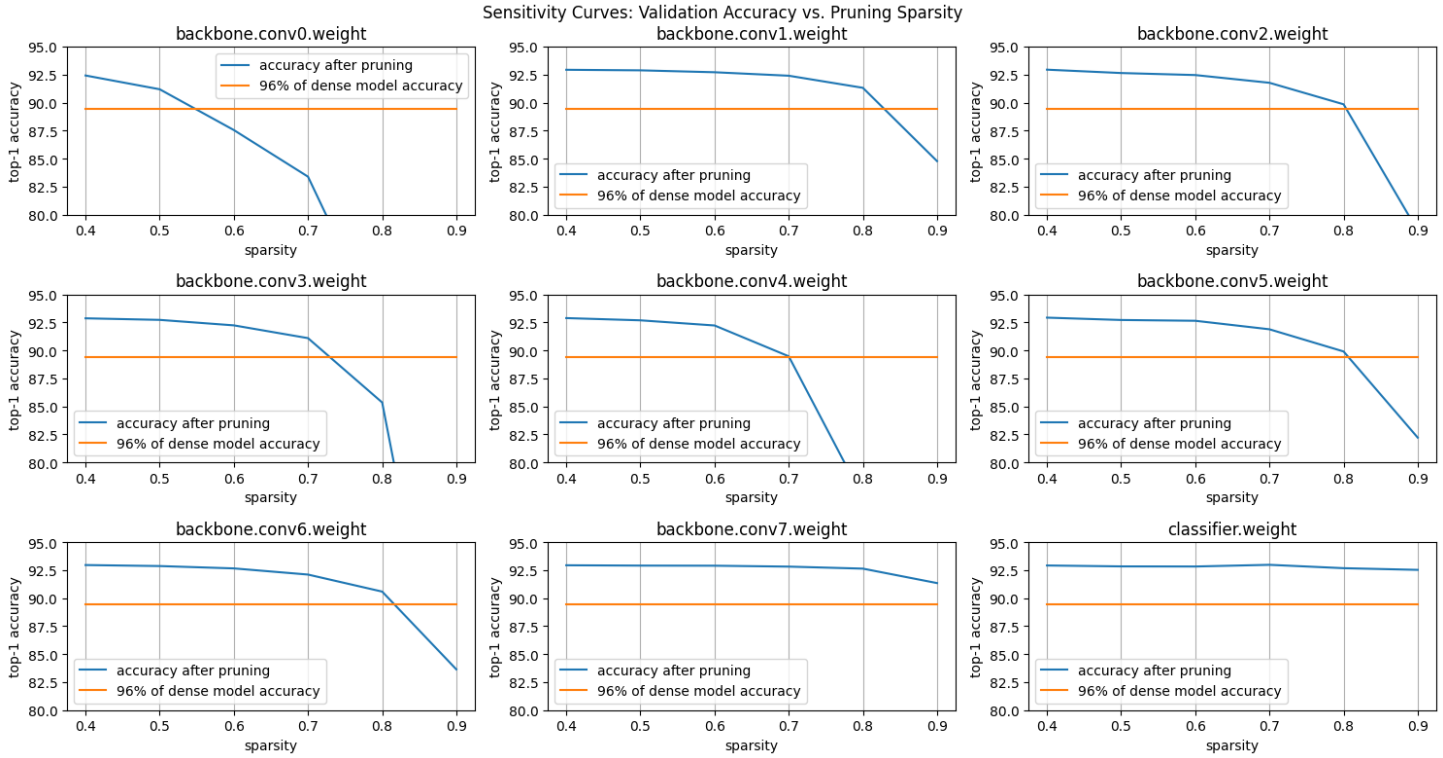


- **Global Magnitude Pruning**

A simple global pruning method that performs pruning using a single global threshold

Sensitivity analysis

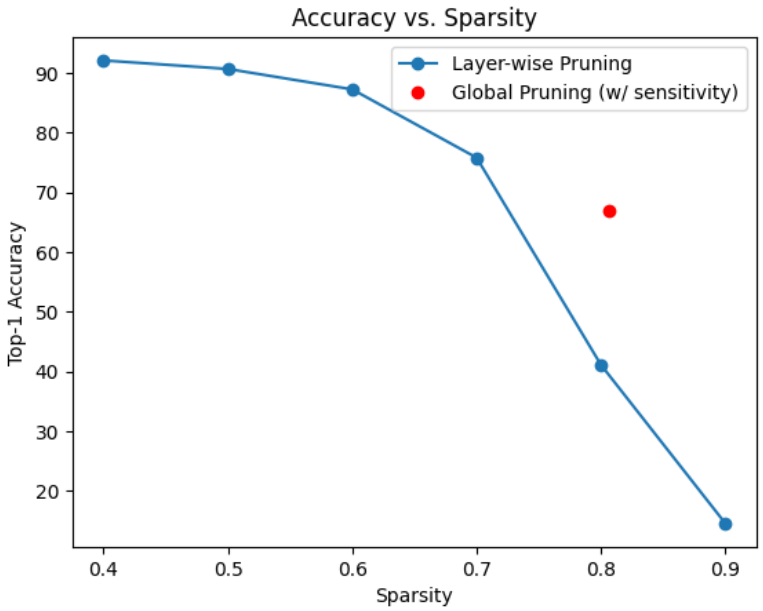
- **Analyzes the sensitivity of the pre-trained model by plotting validation accuracy versus pruning sparsity for various layers**



[실습 2] Sensitivity analysis를 통한 pruning 수행

- Based on the previous sensitivity analysis, assign an appropriate sparsity to each layer and evaluate

- Compare the accuracy with layer-wise pruning that uses uniform sparsity



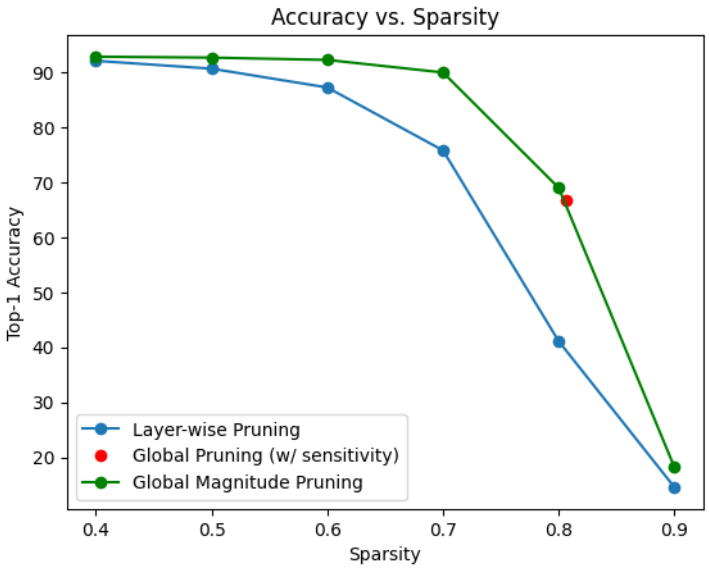
[실습 2] Answer


```
custom_sparsity_dict = {
##################### YOUR CODE STARTS HERE #####################
    # please modify the sparsity value of each layer
    # please DO NOT modify the key of sparsity_dict
    'backbone.conv0.weight': 0.5,
    'backbone.conv1.weight': 0.8,
    'backbone.conv2.weight': 0.8,
    'backbone.conv3.weight': 0.7,
    'backbone.conv4.weight': 0.7,
    'backbone.conv5.weight': 0.8,
    'backbone.conv6.weight': 0.8,
    'backbone.conv7.weight': 0.9,
    'classifier.weight': 0.9
##################### YOUR CODE ENDS HERE #######################
}
```

[실습 3] Global magnitude pruning 구현

- Implement global magnitude pruning, which prunes using a single global threshold

- Compare the accuracy with layer-wise pruning and the global pruning derived from the sensitivity analysis

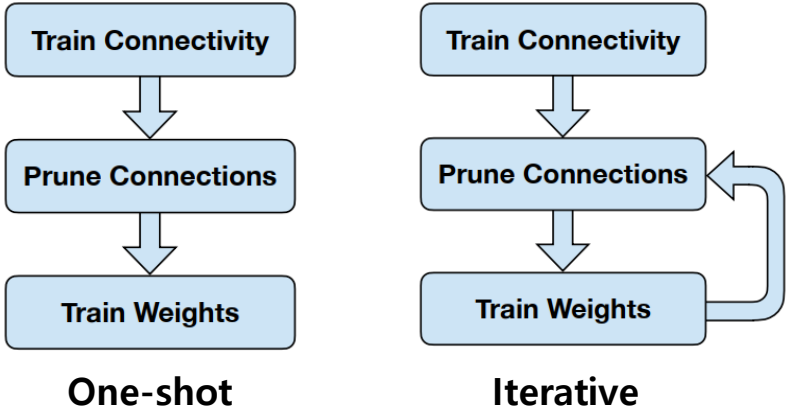


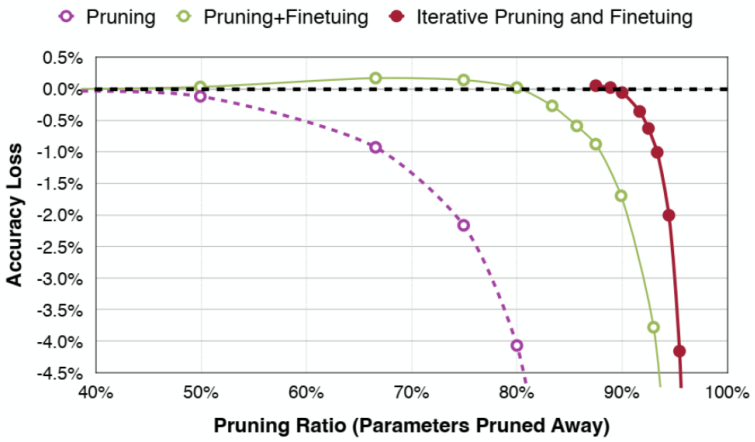
[실습 3] Answer

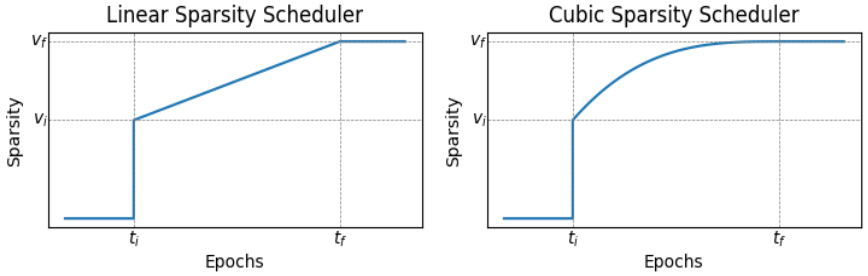

```
##################### YOUR CODE STARTS HERE #####################
# 모든 weight를 하나로 합쳐주세요.
# hint: torch.cat()을 사용하세요.
all_weights = torch.cat(parameters_to_prune)

# all_weights를 기준으로 global threshold를 구해주세요.
num_elements = all_weights.numel()
num_zeros = round(num_elements * sparsity)
importance = torch.abs(all_weights)
threshold = torch.kthvalue(importance, num_zeros)[0]
##################### YOUR CODE ENDS HERE #######################
```

1.3. Pruning Schedule: Overview

- **One-shot Pruning**
- **Iterative Pruning**
  - Sparsity scheduling







Model Training Function


```python
def train(
    model: torch.nn.Module,
    dataloader: DataLoader,
    criterion: torch.nn.Module,
    optimizer: torch.optim.Optimizer,
    scheduler: torch.optim.lr_scheduler.LRScheduler,
    callbacks=None,
) -> None:
    model.train()
    for inputs, targets in tqdm(dataloader, desc='Train', leave=False):
        if torch.cuda.is_available():
            inputs, targets = inputs.cuda(), targets.cuda()
        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, targets)
        loss.backward()
        optimizer.step()
        scheduler.step()
        if callbacks is not None:
            for callback in callbacks:
                callback()
```

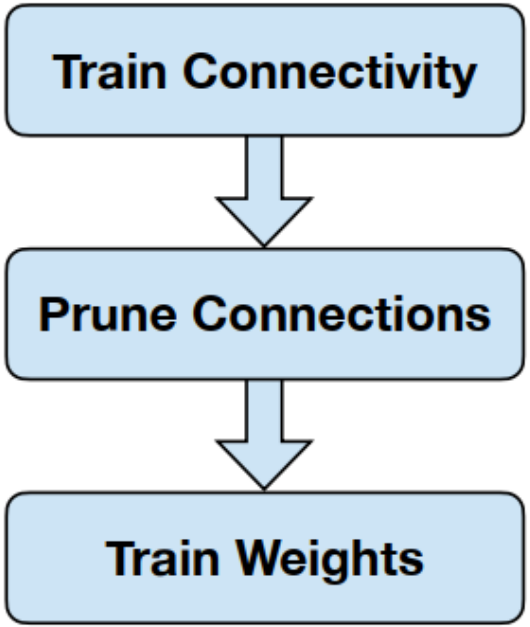
- **Criterion:**
  - Loss function
- **Optimizer:**
  - Algorithm (e.g., SGD) that updates model parameters to advance training
- **Scheduler:**
  - Learning rate scheduler

initializing accumulated gradients to 0

calculating loss and gradients via backward propagation

updating parameters and learning rates

One-shot pruning

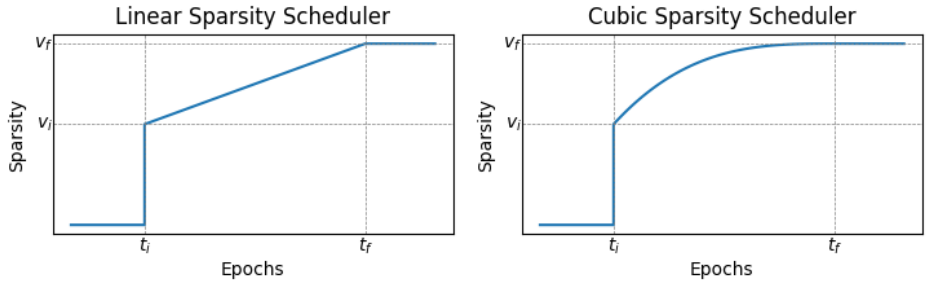


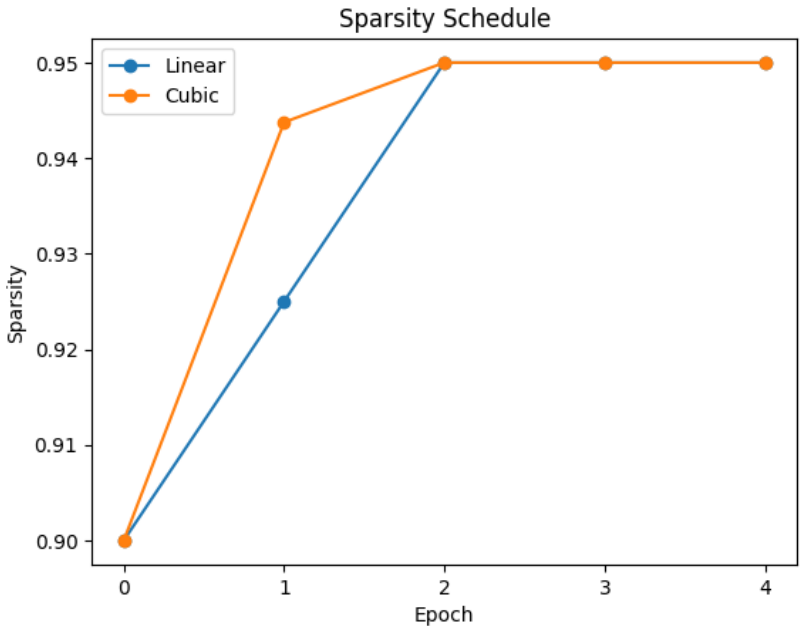

```
Epoch 1/5: accuracy=89.84%

Epoch 2/5: accuracy=90.51%

Epoch 3/5: accuracy=90.88%

Epoch 4/5: accuracy=90.89%

Epoch 5/5: accuracy=90.94%
```

Training accuracy may vary depending on the reading order of the training dataset and the pre-processing (augmentation) methods used

[실습 4] Sparsity scheduler 구현

- **Sparsity scheduler**
  - $v^{(t)} = v_f + (v_i - v_f)\left(1 - \frac{t - t_i}{t_f - t_i}\right)^E$
    - E=1: linear
    - E=3: cubic

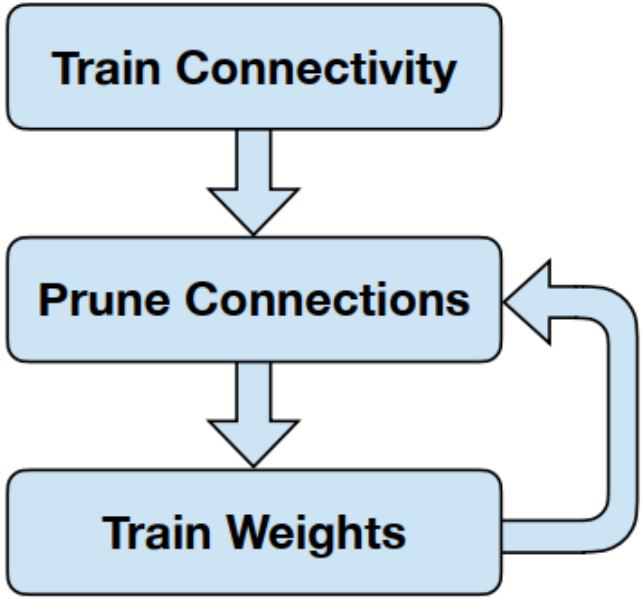




[실습 4] Answer


```python
##################### YOUR CODE STARTS HERE #####################
for epoch in range(num_epochs):
    # 위 수식에 맞게 sparsity를 계산하세요. (단, epoch_start보다 작은 경우 0으로 epochs_end 이후는 sparsity_end로 설정합니다.)
    if epoch < epoch_start:
        sparsity = 0
    elif epoch >= epoch_end:
        sparsity = sparsity_end
    else:
        sparsity = sparsity_end + (sparsity_start - sparsity_end) * (1 - (epoch - epoch_start) / (epoch_end - epoch_start)) ** exponent

    sparsity_schedule.append(sparsity)
##################### YOUR CODE ENDS HERE #######################
```

Iterative pruning



Linear Sparsity Scheduler

```
Epoch 1/5: accuracy=91.68%
Epoch 2/5: accuracy=91.68%
Epoch 3/5: accuracy=90.86%
Epoch 4/5: accuracy=91.03%
Epoch 5/5: accuracy=90.97%
```

Cubic Sparsity Scheduler

```
Epoch 1/5: accuracy=91.66%
Epoch 2/5: accuracy=91.00%
Epoch 3/5: accuracy=91.19%
Epoch 4/5: accuracy=91.41%
Epoch 5/5: accuracy=91.36%
```

Training accuracy may vary depending on the reading order of the training dataset and the pre-processing methods